

COMPLETE DEVELOPER HANDBOOK

REST API

Full Notes — Beginner to Pro, Explained Simply

Beginner Basics

Intermediate Design

Pro-Level Practices

Table of Contents

What is an API & REST?

Beginner

- 1.1 What is an API
- 1.2 What is REST
- 1.3 Client-Server Model

Core REST Principles

Beginner

- 2.1 The 6 REST Constraints
- 2.2 Resources & URIs

HTTP Methods (Verbs)

Beginner

HTTP Status Codes

Beginner

Request & Response Anatomy

Beginner

Headers Explained

Beginner

Designing Good REST URLs

Intermediate

Query Parameters, Filtering, Pagination, Sorting

Intermediate

Request/Response Body Design (JSON)

Intermediate

Authentication & Authorization

Intermediate

Versioning Strategies

Intermediate

Error Handling Best Practices

Intermediate

Idempotency & Safety

Intermediate

Caching in REST APIs

Advanced

Rate Limiting & Throttling

Advanced

HATEOAS

Advanced

Security Best Practices

Advanced

REST vs SOAP vs GraphQL vs gRPC

Advanced

Testing REST APIs

Advanced

Documentation (OpenAPI/Swagger)

Advanced

Scaling & Performance Tips

Advanced

Common Interview Questions

Pro

Quick Reference Cheat Sheet

Pro

1. What is an API & REST?

1.1 What is an API

API stands for **Application Programming Interface**. It is simply a set of rules that lets two software applications talk to each other.

Think of an API like a waiter in a restaurant. You (the client) don't go into the kitchen (the server) yourself. You tell the waiter (API) what you want, the waiter takes your request to the kitchen, and brings back the food (data/response).

1.2 What is REST

REST stands for **Representational State Transfer**. It is an architectural style (a set of rules/guidelines) for designing networked applications — not a strict protocol or a library.

An API that follows REST rules is called a **RESTful API**. It uses standard HTTP methods (GET, POST, PUT, DELETE, etc.) to perform operations on **resources** (like users, products, orders) which are identified by URLs.

1.3 Client-Server Model

REST is built on a simple client-server relationship:

- **Client:** The one who requests data (browser, mobile app, Postman, another server).
- **Server:** The one who has the data/resources and responds to requests.

Flow: Client sends HTTP Request → Server processes it → Server sends HTTP Response (usually JSON)

2. Core REST Principles

2.1 The 6 REST Constraints

For an API to be truly "RESTful," it should follow these 6 architectural constraints:

Constraint	Meaning in Simple Words
1. Client-Server	Frontend and backend are separate and can evolve independently.
2. Statelessness	Server does NOT remember anything about the client between requests. Every request must contain all the info needed.
3. Cacheable	Responses must define themselves as cacheable or not, to improve performance.
4. Uniform Interface	Consistent way to access resources — same rules everywhere (URLs, verbs, formats).
5. Layered System	Client can't tell if it's talking directly to the server or through a proxy/load balancer.
6. Code on Demand (optional)	Server can send executable code (like JS) to the client if needed. Rarely used.

Interview Tip: "Statelessness" is the most commonly asked constraint. Remember: the server never stores client session info between requests — the client must send auth tokens, etc., every single time.

2.2 Resources & URIs

In REST, everything is treated as a **resource** — a "thing" you can act upon. Examples: a user, a product, an order.

Each resource gets a unique URL, called a **URI** (Uniform Resource Identifier).

Good URI (Resource-based)	Bad URI (Action-based)
GET /users	GET /getAllUsers
GET /users/23	GET /getUserById?id=23
POST /users	POST /createNewUser

Rule: URLs should represent *nouns* (resources), never *verbs* (actions). The HTTP method already tells the action.

3. HTTP Methods (Verbs)

These are the "actions" you can perform on a resource:

Method	Purpose	Safe?	Idempotent?	Example
GET	Read/fetch data	Yes	Yes	GET /users/5
POST	Create new resource	No	No	POST /users
PUT	Update/replace entire resource	No	Yes	PUT /users/5
PATCH	Partially update resource	No	No*	PATCH /users/5
DELETE	Remove a resource	No	Yes	DELETE /users/5

Safe = doesn't change server data. **Idempotent** = calling it multiple times gives the same result as calling it once. (*PATCH is technically not guaranteed idempotent, though often implemented that way.)

PUT vs PATCH — the classic confusion

- **PUT** replaces the *whole* object. If you leave out a field, it may get wiped/set to null.
- **PATCH** updates only the fields you send. Rest stay untouched.

4. HTTP Status Codes

Status codes tell the client what happened with their request. They are grouped into 5 classes:

1xx – Informational

Rarely used directly by API developers (e.g., 100 Continue).

2xx – Success

Code	Meaning	When to Use
200 OK	Success	Standard success for GET, PUT, PATCH
201 Created	Resource created	After successful POST
202 Accepted	Request accepted, processing async	Long-running background jobs
204 No Content	Success, no body returned	After DELETE, or PUT with no response body

3xx – Redirection

Code	Meaning
301 Moved Permanently	Resource URL changed permanently
304 Not Modified	Cached version is still valid (used with ETags)

4xx – Client Errors

Code	Meaning	When to Use
400 Bad Request	Invalid input/syntax	Malformed JSON, missing required field
401 Unauthorized	Not authenticated	Missing/invalid token
403 Forbidden	Authenticated but not allowed	User lacks permission
404 Not Found	Resource doesn't exist	Invalid ID/URL
405 Method Not Allowed	Wrong HTTP method used	DELETE on read-only endpoint
409 Conflict	Conflicting state	Duplicate entry, version conflict
422 Unprocessable Entity	Validation failed	Data is well-formed but invalid (e.g. email format wrong)
429 Too Many Requests	Rate limit exceeded	Client is calling API too fast

5xx – Server Errors

Code	Meaning
500 Internal Server Error	Generic server crash/bug
502 Bad Gateway	Upstream server sent invalid response
503 Service Unavailable	Server overloaded or under maintenance
504 Gateway Timeout	Upstream server took too long

Common Mistake: Beginners often return 200 OK for everything, even errors, with an "error" field in the JSON body. This breaks REST conventions and makes debugging/monitoring harder. Always use proper status codes.

5. Request & Response Anatomy

Anatomy of an HTTP Request

```
POST /api/v1/users HTTP/1.1
Host: example.com
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

{
  "name": "Riya Sharma",
  "email": "riya@example.com"
}
```

- **Request Line:** Method + Path + HTTP version
- **Headers:** Metadata (content type, auth token, etc.)
- **Body:** The actual data being sent (for POST/PUT/PATCH)

Anatomy of an HTTP Response

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/v1/users/45

{
  "id": 45,
  "name": "Riya Sharma",
  "email": "riya@example.com",
  "createdAt": "2026-07-09T10:15:00Z"
}
```


6. Headers Explained

Header	Purpose
Content-Type	Format of the request/response body (e.g. <code>application/json</code>)
Accept	What format the client wants back
Authorization	Credentials — e.g. <code>Bearer <token></code>
Cache-Control	Caching rules (e.g. <code>no-cache</code> , <code>max-age=3600</code>)
ETag	A version identifier for a resource, used for caching/conditional requests
User-Agent	Info about the client app/browser making the request
X-RateLimit-Remaining	Custom header often used to show remaining API calls allowed

7. Designing Good REST URLs

Good URL design is what separates a beginner API from a professional one.

- Use **plural nouns**: `/users` not `/user`
- Use **nesting** for relationships: `/users/23/orders` (orders of user 23)
- Avoid deep nesting beyond 2 levels: prefer `/orders/99/items` over `/users/23/orders/99/items/5`
- Use **hyphens**, not underscores or camelCase, in URLs: `/user-profiles`
- Keep URLs lowercase
- Don't include file extensions like `.json` in the URL
- Never use verbs in the path — the HTTP method is the verb

Action	REST-ful URL
Get all products	<code>GET /products</code>
Get one product	<code>GET /products/12</code>
Create a product	<code>POST /products</code>
Update a product	<code>PUT /products/12</code>
Delete a product	<code>DELETE /products/12</code>
Get reviews of a product	<code>GET /products/12/reviews</code>

8. Query Parameters, Filtering, Pagination, Sorting

Query parameters (after `?`) let clients customize what they get back, without changing the core resource URL.

Filtering

```
GET /products?category=shoes&color=red
```

Sorting

```
GET /products?sort=price           (ascending)
GET /products?sort=-price          (descending, using minus sign)
```

Pagination

```
GET /products?page=2&limit=20
GET /products?offset=40&limit=20
```

Best practice response for paginated data:

```
{
  "data": [ ... ],
  "meta": {
    "page": 2,
    "limit": 20,
    "totalItems": 134,
    "totalPages": 7
  }
}
```

Searching

```
GET /products?search=running+shoes
```

Field Selection (Sparse Fieldsets)

```
GET /products?fields=id,name,price
```

9. Request/Response Body Design (JSON)

JSON (JavaScript Object Notation) is the most common data format used in REST APIs today (replacing older XML-based APIs).

Good Practices

- Use **camelCase** for JSON keys: `firstName`, not `first_name` (though `snake_case` is also acceptable if consistent)
- Always wrap collections in a consistent structure (e.g. `{"data": [...]}`)
- Use ISO 8601 for dates: `"2026-07-09T10:15:00Z"`
- Keep responses flat where possible; avoid deeply nested objects
- Never expose internal database fields (like password hashes, internal IDs) in responses

Example: Well-designed Response

```
{
  "data": {
    "id": 45,
    "name": "Riya Sharma",
    "email": "riya@example.com",
    "role": "admin"
  },
  "meta": {
    "requestId": "a1b2c3"
  }
}
```

10. Authentication & Authorization

Authentication = Who are you? **Authorization** = What are you allowed to do?

Common Authentication Methods

Method	How it Works	Use Case
Basic Auth	Username:password encoded in Base64, sent in header	Simple internal tools (not very secure alone)
API Key	Static key sent in header or query param	Public APIs, simple integrations
Bearer Token (JWT)	Signed token proves identity, sent as <code>Authorization: Bearer <token></code>	Most modern web/mobile apps
OAuth 2.0	Delegated authorization via a trusted provider (Google, GitHub login)	"Login with Google" type flows
Session Cookies	Server stores session state; cookie sent with each request	Traditional server-rendered web apps

JWT Structure (Beginner-Friendly)

A JWT (JSON Web Token) has 3 parts separated by dots: `header.payload.signature`

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJQ1fQ.4f8s9d...
```

- **Header:** Algorithm used to sign the token
- **Payload:** The actual claims/data (e.g. userId, role, expiry)
- **Signature:** Ensures the token wasn't tampered with

Pro Tip: Never store sensitive data (like passwords) in a JWT payload — it's only encoded, not encrypted, and can be decoded by anyone.

11. Versioning Strategies

APIs evolve. Versioning prevents breaking existing clients when you make changes.

Strategy	Example	Pros / Cons
URI Versioning	<code>/api/v1/users</code>	Most common, easy to see, but clutters URL
Header Versioning	<code>Accept: application/vnd.myapi.v2+json</code>	Clean URLs, but harder to test in browser
Query Param Versioning	<code>/users?version=2</code>	Simple but easy to forget/misuse

Industry Standard: URI versioning (`/v1/` , `/v2/`) is the most widely adopted because it's explicit and simple to route.

12. Error Handling Best Practices

A good error response tells the client exactly what went wrong and how to fix it.

Well-Structured Error Response

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The 'email' field must be a valid email address.",
    "field": "email",
    "requestId": "a1b2c3"
  }
}
```

- Always match the correct HTTP status code with the error
- Use consistent error format across the whole API
- Never leak stack traces or internal server details to the client
- Include a machine-readable error `code` along with a human-readable `message`

13. Idempotency & Safety

This is a favorite topic in senior-level interviews.

- **Safe methods** (GET, HEAD, OPTIONS): Never change server state, no matter how many times called.
- **Idempotent methods** (GET, PUT, DELETE): Calling once or a hundred times produces the same end result on the server.
- **Non-idempotent**: POST — calling it twice usually creates two resources.

Real-World Use: Idempotency Keys

For payment APIs, even POST requests need idempotency (to avoid double-charging on network retry). Solution: client sends a unique **Idempotency-Key** header; server stores it and returns the same result if the same key is seen again.

```
POST /payments
Idempotency-Key: 8f14e45f-ceed-4c...
```


14. Caching in REST APIs

Caching reduces server load and speeds up responses by reusing previous results.

Cache-Control Header

```
Cache-Control: max-age=3600, public
```

ETags (Entity Tags) — Conditional Requests

An ETag is like a fingerprint/hash of the resource's current state.

1. Server sends `ETag: "abc123"` with the response
2. Client stores it, and next time sends `If-None-Match: "abc123"`
3. If nothing changed, server replies `304 Not Modified` (no body sent — saves bandwidth)

15. Rate Limiting & Throttling

Rate limiting protects your API from abuse and overload by capping how many requests a client can make in a time window.

Common Response Headers

Header	Meaning
X-RateLimit-Limit	Max requests allowed in the window
X-RateLimit-Remaining	Requests left in current window
X-RateLimit-Reset	When the limit resets (timestamp)
Retry-After	Seconds to wait before retrying (sent with 429)

Common Algorithms

- **Fixed Window:** Simple counter reset every X seconds
- **Sliding Window:** More accurate, tracks a rolling time window
- **Token Bucket:** Tokens refill at a steady rate; each request consumes one
- **Leaky Bucket:** Requests processed at a constant fixed rate, extras queued or dropped

16. HATEOAS

HATEOAS = **H**ypermedia **A**s **T**he **E**ngine **O**f **A**pplication **S**tate. It's an advanced REST constraint (part of "Uniform Interface") where the API response includes links to related actions, so the client doesn't need to hardcode URLs.

```
{
  "id": 45,
  "name": "Riya Sharma",
  "_links": {
    "self": { "href": "/users/45" },
    "orders": { "href": "/users/45/orders" },
    "delete": { "href": "/users/45", "method": "DELETE" }
  }
}
```

Very few real-world APIs implement full HATEOAS because it adds complexity — but it's important to know for interviews and for truly "Level 3" REST maturity (Richardson Maturity Model).

Richardson Maturity Model (Pro Concept)

Level	Description
Level 0	Single URL, everything via POST (like RPC/SOAP)
Level 1	Multiple resource-based URLs, but still mostly one HTTP method
Level 2	Proper use of HTTP methods + status codes (most real-world "REST" APIs stop here)
Level 3	Full HATEOAS — true REST

17. Security Best Practices

- **Always use HTTPS** — never send tokens/passwords over plain HTTP
- **Validate & sanitize all input** to prevent SQL injection, NoSQL injection, XSS
- **Use short-lived access tokens** + refresh tokens instead of long-lived tokens
- **Never expose sensitive data** (passwords, secrets, internal IDs) in responses
- **Implement rate limiting** to prevent brute-force and DDoS abuse
- **Use CORS properly** — only allow trusted origins
- **Apply the Principle of Least Privilege** — users/services get only the access they need
- **Log and monitor** suspicious activity (failed logins, unusual traffic spikes)
- **Set proper security headers:** `Content-Security-Policy` , `X-Content-Type-Options` , `Strict-Transport-Security`

OWASP API Security Top Risks to know: Broken Object Level Authorization (BOLA), Broken Authentication, Excessive Data Exposure, Lack of Rate Limiting, Mass Assignment, Security Misconfiguration.

18. REST vs SOAP vs GraphQL vs gRPC

Feature	REST	SOAP	GraphQL	gRPC
Data Format	JSON (mostly)	XML only	JSON	Protocol Buffers (binary)
Protocol	HTTP	HTTP, SMTP, etc.	HTTP (single endpoint)	HTTP/2
Flexibility	Fixed structure per endpoint	Rigid, strict contracts	Client asks for exactly the fields it needs	Strict schema (proto files)
Best For	Public web/mobile APIs	Enterprise, banking, legacy systems	Complex, nested data needs	Microservice-to-microservice, high performance
Speed	Good	Slower (heavy XML)	Good	Very fast (binary + HTTP/2)

19. Testing REST APIs

Manual Testing Tools

- **Postman** — most popular GUI tool for sending requests, saving collections, writing test scripts
- **cURL** — command-line tool, great for quick tests and scripts
- **Insomnia / Thunder Client** — lightweight alternatives to Postman

Example cURL Command

```
curl -X POST https://api.example.com/v1/users \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"name": "Riya", "email": "riya@example.com"}'
```

Automated Testing

- **Unit tests:** Test individual functions/logic (Jest, PyTest, JUnit)
- **Integration tests:** Test full request → response flow (Supertest, RestAssured)
- **Contract tests:** Ensure API matches its documented schema (Pact)
- **Load tests:** Check performance under stress (k6, JMeter, Locust)

20. Documentation (OpenAPI/Swagger)

OpenAPI Specification (OAS), formerly known as Swagger, is the industry standard for describing REST APIs in a machine-readable format (YAML/JSON). It allows auto-generated docs, client SDKs, and interactive "try it out" pages.

```
openapi: 3.0.0
info:
  title: User API
  version: 1.0.0
paths:
  /users/{id}:
    get:
      summary: Get a user by ID
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Successful response
```

Tools like Swagger UI, Redoc, and Postman can auto-generate interactive documentation directly from an OpenAPI YAML/JSON file.

21. Scaling & Performance Tips

- Use **pagination** — never return unbounded large lists
- Add **database indexing** on frequently queried fields
- Use a **CDN** for static/cacheable responses
- Implement **load balancing** across multiple server instances
- Use **connection pooling** for databases
- Apply **compression** (gzip/brotli) on responses
- Consider **asynchronous processing** (queues) for slow/heavy tasks — respond **202 Accepted** immediately
- Monitor with tools like Prometheus, Grafana, New Relic, Datadog

22. Common Interview Questions

Q1: What makes an API "RESTful"?

It follows REST's architectural constraints — statelessness, uniform interface, resource-based URLs, standard HTTP verbs, and client-server separation.

Q2: Difference between PUT and PATCH?

PUT replaces the whole resource; PATCH updates only specified fields.

Q3: What is statelessness and why does it matter?

Server stores no client session data between requests — this makes REST APIs easier to scale horizontally since any server instance can handle any request.

Q4: How do you version an API without breaking existing clients?

Use URI versioning (`/v1/` , `/v2/`) so old clients keep hitting the old version while new clients move to the new one.

Q5: What's the difference between 401 and 403?

401 = not authenticated (who are you?). 403 = authenticated but not authorized (you can't do this).

Q6: How would you secure a public REST API?

HTTPS, token-based auth (JWT/OAuth2), rate limiting, input validation, proper CORS, least-privilege access control.

Q7: Why is GET considered both "safe" and "idempotent"?

It never changes server state (safe), and calling it repeatedly always returns the same result without side effects (idempotent).

23. Quick Reference Cheat Sheet

Concept	Key Takeaway
REST	Architectural style using HTTP + resources, not a strict protocol
Statelessness	No session stored server-side between requests
GET/POST/PUT/PATCH/DELETE	Read / Create / Replace / Partial Update / Remove
2xx / 4xx / 5xx	Success / Client error / Server error
JWT	Self-contained signed token for stateless auth
Pagination	Use <code>page / limit</code> or <code>offset / limit</code> to avoid huge payloads
Idempotency	Same request repeated = same result (GET, PUT, DELETE)
Versioning	Prefer <code>/v1/</code> URI-based versioning
HATEOAS	Response includes links to related actions (true REST, Level 3)
OpenAPI/Swagger	Standard for documenting & auto-generating API docs

You've covered REST APIs from the ground up to professional-level design, security, and interview prep.
Practice by building a small CRUD API (e.g., a "Books" or "Tasks" API) applying everything in this handbook — that's the fastest way to make it stick.